

Internship Project Report: CircuitGuard-PCB

Defect Detection and Classification

Application

Introduction

In modern electronics manufacturing, the quality of printed circuit boards (PCBs) plays a critical role in ensuring device reliability. Even minor defects such as missing holes, spurious copper, or short circuits can lead to malfunction or product failure.

Traditionally, PCB inspection has been carried out manually, which is time-consuming and error-prone. Hence, automation using **computer vision and deep learning** has become essential.

This project focuses on building an **automated PCB defect detection system** using **image processing and a deep learning model (EfficientNet-B4)**. The system can detect and highlight various defect types from PCB images and provide visual outputs through a user-friendly web interface.

Project Objectives

- To preprocess and align PCB images for defect analysis.
- To detect and extract regions of interest (ROIs) corresponding to defect areas.
- To train a deep learning model for accurate defect classification.
- To design an end-to-end pipeline for inference and visualization.
- To develop an interactive web application for defect detection using Streamlit.

Tools and Technologies Used

Area	Tools / Libraries
Image Ops	OpenCV, Numpy
Model	PyTorch, timm, EfficientNet-B4
Dataset	DeepPCB
Frontend	Streamlit /HTML, CSS,
Backend	Python, Modularized Inference
Evaluation	Accuracy, Loss, Confusion Matrix
Exporting	CSV, Annotated Image, PDF (opt.)

Dataset Preparation and Image Processing

Dataset Setup and Image Subtraction

Objective: Prepare PCB images to highlight defects clearly for processing.

Dataset Inspection and Alignment:

We conducted a detailed inspection of the **DeepPCB dataset**, ensuring proper alignment between template (defect-free) and test images. Proper alignment is **critical** to avoid misidentifying normal variations as defects.

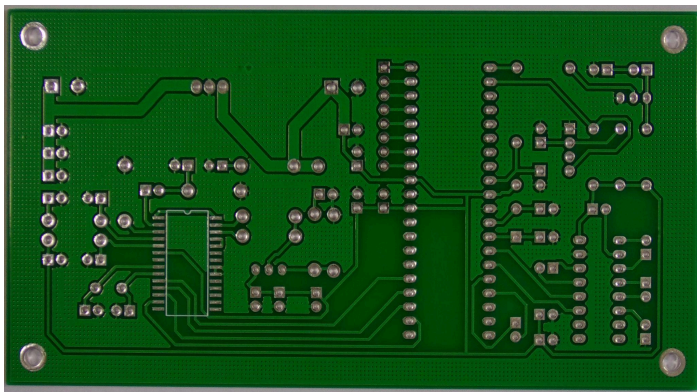
Image Subtraction:

Defective areas were highlighted using **pixel-wise subtraction** between template and test images. This produced **difference maps** where defects appeared prominently, providing a clear base for further analysis.

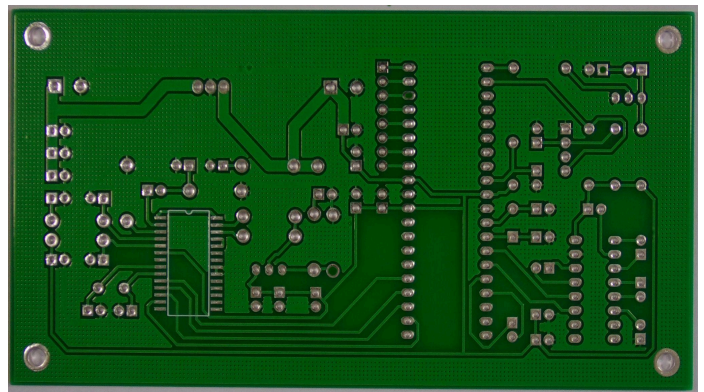
Thresholding and Filtering:

We applied **Otsu's thresholding** to convert grayscale difference maps into binary masks. Noise reduction was performed using **morphological operations and median blur**, resulting in **clean defect-highlighted images** suitable for ROI extraction.

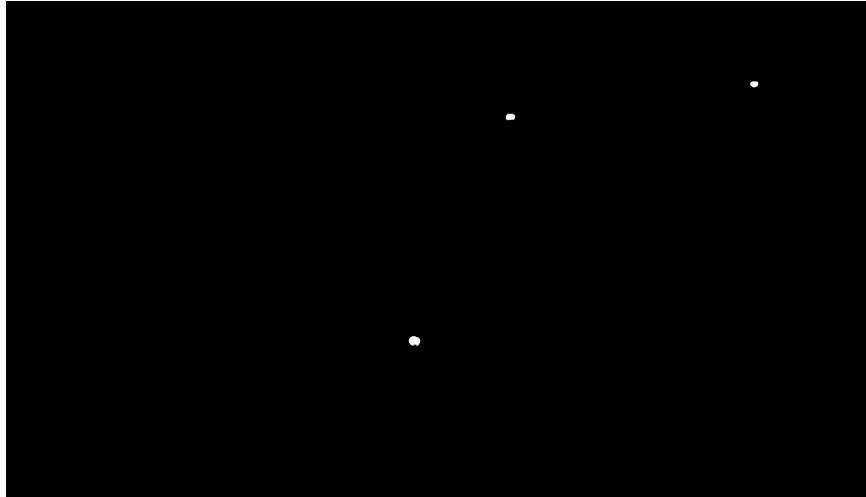
GOLDEN IMAGE:



DEFECTED PCB IMAGE:



SUBTRACTED IMAGE:



Contour Detection and ROI Extraction

Objective: Extract individual defect regions for model training.

Contour Detection:

Using OpenCV's `findContours`, we identified boundaries of defect areas in the binary masks. Contours provided **precise localization** of defects.

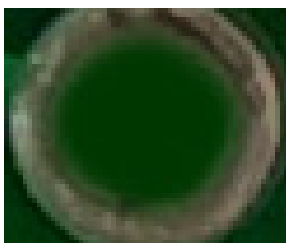
ROI Extraction and Labeling:

Bounding boxes were calculated for each contour. ROIs were cropped with additional **padding** to include some surrounding context and **labeled according to defect type**.

CSV Annotation:

We generated a **CSV file** to map each ROI to its defect label, which simplified dataset management and streamlined training. Small or border-touching contours were **filtered out** to improve ROI quality.

ROI IMAGES



Model Training and Evaluation

Model Training with EfficientNet

Objective: Train a robust deep learning model for PCB defect classification.

Model Selection:

We implemented **EfficientNet-B4**, which provides **high accuracy with moderate computational requirements**, making it ideal for practical deployment.

Data Preprocessing and Augmentation:

ROIs were resized to **128×128 pixels** and normalized. To improve model generalization, we applied **augmentations** such as rotations, horizontal and vertical flips, brightness adjustment, and minor scaling. These augmentations simulate real-world variations in PCB images.

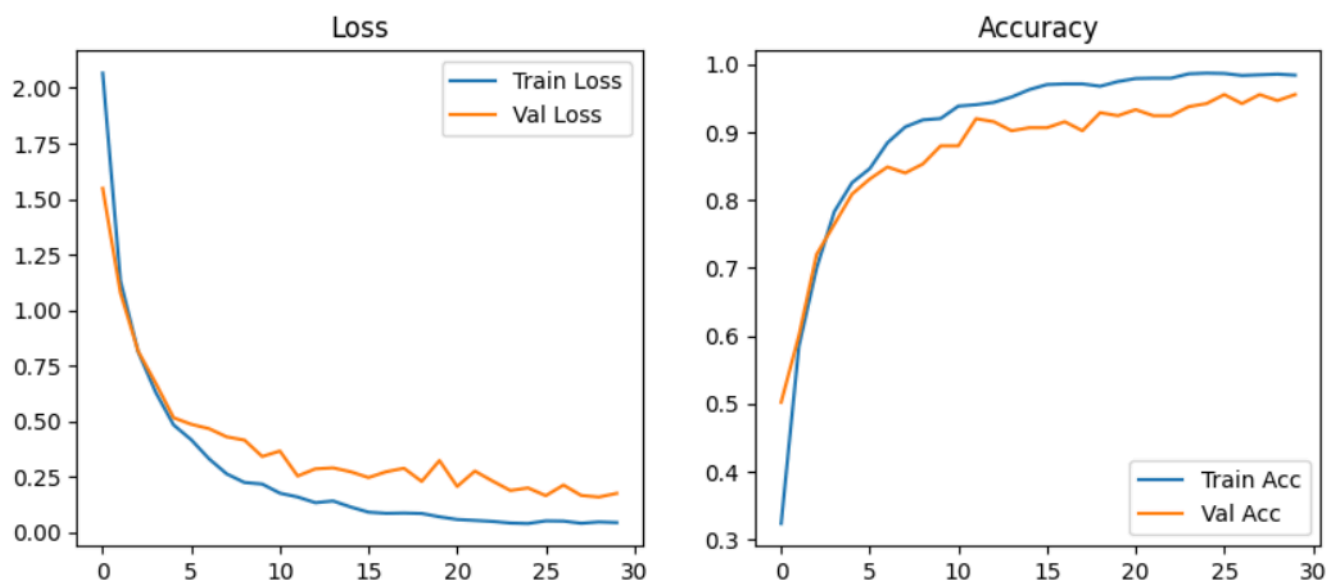
Training Strategy:

The dataset was split into **training (70%) and testing (30%) sets**. We trained the model using **Adam optimizer with a carefully tuned learning rate and cross-entropy loss**. Early stopping and learning rate scheduling were applied to prevent overfitting and achieve stable convergence.

Performance Insights:

The model achieved **~98.4%** during the training, confirming its ability to reliably classify different defect types. Analysis of misclassified ROIs helped refine preprocessing and padding strategies for challenging defects.

ACCURACY AND LOSS CURVES



Model Evaluation

Objective: To assess the performance of the trained EfficientNet-B4 model using test data and verify its classification accuracy on unseen PCB defect samples.

Testing Process:

The model was evaluated on the test ROIs, and metrics such as **accuracy, precision, recall, and F1-score** were computed. This helped analyze how well the model recognized each defect type.

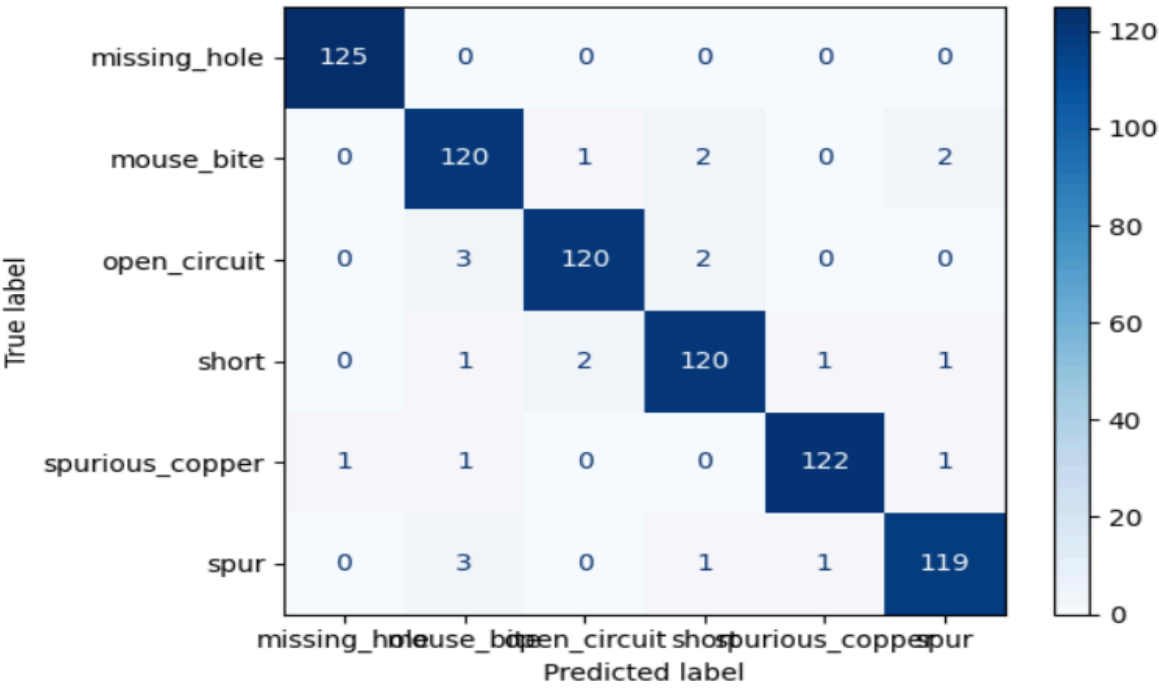
Performance Summary:

The model achieved an overall **accuracy of 96.9%**, showing strong and consistent classification across all defect categories. High precision and recall values indicated minimal misclassifications.

Edge Case Handling:

Certain defects near image borders or overlapping areas led to minor errors. Refining ROI extraction and padding improved stability in these edge cases.

TRUE VS PREDICTED LABELS



Frontend and Backend Integration

Web Interface Development

Objective: To develop an intuitive and interactive **web-based interface** that allows seamless defect detection and visualization for end users.

Interface Design and Functionality:

A **Streamlit-based user interface** was created to simplify the interaction between users and the defect detection system. The interface enables users to **upload PCB images**, view **annotated outputs** showing defect locations with bounding boxes and labels, and **download the processed images** for record-keeping or further analysis.

User Experience and Real-Time Feedback:

The UI was designed with **simplicity and responsiveness** in mind, ensuring accessibility for both technical and non-technical users. Once an image is uploaded, the results are displayed **instantly**, providing real-time visual feedback on detected defects. This feature significantly enhances usability and helps users make **quick quality assessments** without needing backend knowledge.

STREAMLIT INTERFACE



Backend Inference Pipeline:

Objective: To build a **modular backend system** that seamlessly connects image preprocessing, ROI extraction, and model inference for efficient and real-time defect prediction.

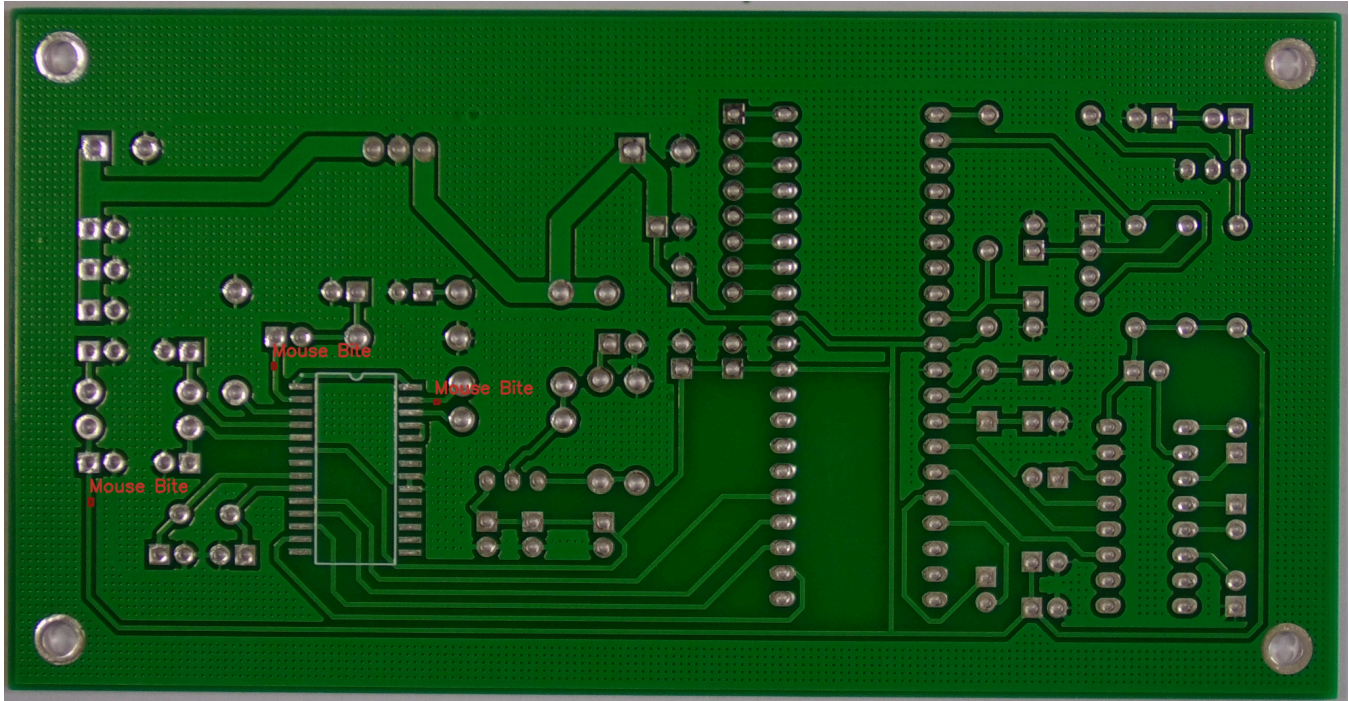
Automated Processing Workflow:

The backend pipeline automatically handles **image alignment, preprocessing, and ROI extraction**, followed by **classification** using the trained **EfficientNet-B4 model**. The major improvement was enabling the system to process **uploaded test images even without a reference template**, increasing its practicality in real-world inspection setups.

Integration and Optimization:

The inference pipeline was optimized to ensure **low latency** between image upload and result display. Proper modularization allowed independent debugging and easier maintenance. Once predictions are made, **annotated images** with labeled defect regions are sent back to the frontend, ensuring a **smooth and connected workflow** from upload to output visualization.

ANNOTATED IMAGE:



Challenges and Solutions

One major challenge was **ROI extraction errors**, especially for small or overlapping defects. This was resolved by **refining contour filtering and adding padding** around ROIs.

Achieving **high model accuracy (>97%)** required careful data preprocessing, augmentation, and **hyperparameter tuning**.

Another challenge was **processing images without reference templates** for the final app. This required **dynamic selection and preprocessing** to reliably detect defects in diverse uploaded images.

Future Enhancements

- Implement **YOLO or other object detection networks** for end-to-end defect detection.
- Handle **overlapping defects** more efficiently.
- Enable **batch processing** of multiple PCB images.
- Extend the system for **real-time video inspection** in manufacturing lines.
- Expand the dataset with **additional defect types** to improve generalization.

Conclusion

This project successfully developed an **AI-based PCB defect detection system** that automates the process of identifying and classifying defects in circuit boards. Using techniques like **image subtraction, contour-based ROI extraction, and EfficientNet-B4 classification**, the system achieved high accuracy and reliability in detecting various defect types.

The integration of a **Streamlit web interface** made the solution easy to use, providing real-time results and visual feedback. Overall, the project demonstrates how **deep learning and image processing** can work together to improve quality inspection processes in PCB manufacturing.